

Two-Dimensional Heat Front Position with Wall Effects: Bessel Function Correction for Cylindrical Geometry

Radiation Diffusion Analysis

July 12, 2026

Abstract

This document describes the analytical model for computing the two-dimensional heat front position $z_F(r, t)$ in a cylindrical radiation diffusion problem with wall effects. Starting from the one-dimensional Marshak boundary condition solution, we extend to cylindrical geometry using separation of variables and Bessel function eigenmodes. The radial structure of the heat front is determined by the fundamental Bessel mode $J_0(\kappa_0 r)$, where κ_0 is the first eigenvalue of the albedo-dependent boundary value problem. We describe the complete computational pipeline including eigenvalue calculation, time-marching with Marshak boundary conditions, wall energy loss, and visualization of the curved front structure.

Contents

1	Introduction	3
2	Governing Equations	3
2.1	Radiation Diffusion Equation	3
2.2	Cylindrical Coordinates	3
2.3	Marshak Boundary Condition	4
3	One-Dimensional Solution (Centerline)	4
3.1	Self-Similar Heat Front	4
3.2	Time-Marching Algorithm (Appendix A)	4
4	Wall Energy Loss and Albedo	5
4.1	Energy Balance at Wall	5
4.2	Albedo (Wall Reflectivity)	5
5	Two-Dimensional Extension: Separation of Variables	6
5.1	Assumed Product Solution	6
5.2	Radial Eigenvalue Problem	6
5.3	Bessel Function Eigenmodes	6
5.4	Eigenvalue Solution Strategy	6

6	Radial Front Position	7
6.1	Front Definition	7
6.2	Physical Interpretation	7
7	Two-Dimensional Temperature Distribution	7
7.1	Self-Similar Temperature Profile	7
7.2	Radially-Dependent Temperature Field	8
7.3	Key Features of the Temperature Field	8
7.4	Temperature Heatmap Visualization	8
8	Computational Implementation	9
8.1	Eigenvalue Calculation	9
8.2	Time-Marching with Bessel Data Storage	9
8.3	Wall-Profile Construction in Heatmaps	10
8.4	Foam-Gold Region Differentiation and Temperature Assignment	10
8.5	Shock Front in the Wall Layer	11
8.6	Radial Front Reconstruction	12
9	Results and Visualization	13
9.1	Output Files	13
9.2	Typical Behavior	13
10	Code Structure	14
10.1	Module Organization	14
10.2	Data Flow	14
11	Extensions and Future Work	15
11.1	Higher-Order Modes	15
11.2	Time-Dependent Albedo	15
11.3	Ablation Effects	15
11.4	Effective Lambda Correction	16
12	Validation and Benchmarking	16
12.1	Comparison with Experiments	16
12.2	Code-to-Code Comparison	16
12.3	Convergence Tests	16
13	Conclusion	17
A	Derivation of Eigenvalue Equation	17
B	Numerical Solution of Transcendental Equation	18
C	Bessel Function Properties	18
C.1	Key Identities	18
C.2	Asymptotic Behavior	19

1 Introduction

In cylindrical radiation diffusion problems, the heat front propagates not only axially (along z) but also varies radially (along r) due to wall effects. The one-dimensional solution provides the centerline front position $z_F(t)$, but in reality:

- Wall absorption causes energy loss
- Radial temperature gradients develop
- The front becomes curved in (r, z) space

This document describes how we compute:

$$z_F(r, t) = z_F(t) \cdot J_0(\kappa_0 r) \quad (1)$$

where J_0 is the zeroth-order Bessel function of the first kind, and κ_0 is the first eigenvalue satisfying a transcendental equation involving the wall albedo.

2 Governing Equations

2.1 Radiation Diffusion Equation

The temperature evolution in the foam material is governed by:

$$\frac{\partial U}{\partial t} = \nabla \cdot (D \nabla U) \quad (2)$$

where:

- $U = aT^4$ is the radiation energy density (erg/cm³)
- T is the temperature (in units of energy, typically eV)
- $a = 4\sigma_{SB}/c$ is the radiation constant
- $D = \frac{c\lambda}{3}$ is the radiation diffusion coefficient
- $\lambda = gT^\alpha \rho^{-\lambda_p-1}$ is the Rosseland mean free path

2.2 Cylindrical Coordinates

In axisymmetric cylindrical geometry (r, z) :

$$\frac{\partial U}{\partial t} = \frac{1}{r} \frac{\partial}{\partial r} \left(r D \frac{\partial U}{\partial r} \right) + \frac{\partial}{\partial z} \left(D \frac{\partial U}{\partial z} \right) \quad (3)$$

The domain is:

- Radial: $0 \leq r \leq R$ (cylinder radius)
- Axial: $0 \leq z \leq L$ (cylinder length)

2.3 Marshak Boundary Condition

At the irradiated surface ($z = 0$), the Marshak boundary condition relates incoming and outgoing radiation:

$$\sigma_{SB} T_D^4(t) = \sigma_{SB} T_s^4(t) + \frac{F(0, t)}{2} \quad (4)$$

where:

- $T_D(t)$ is the drive temperature (time-dependent X-ray source)
- $T_s(t)$ is the surface temperature
- $F(0, t) = -D \frac{\partial U}{\partial z} \Big|_{z=0}$ is the heat flux at the surface
- σ_{SB} is the Stefan-Boltzmann constant

Rearranging:

$$F(0, t) = 2\sigma_{SB} (T_D^4(t) - T_s^4(t)) \quad (5)$$

3 One-Dimensional Solution (Centerline)

3.1 Self-Similar Heat Front

For the 1D problem (neglecting radial variation), the heat front position follows:

$$x_F^2(t) = \frac{2 + \epsilon}{1 - \epsilon} \cdot C(\rho) \cdot H(t)^{-\epsilon} \cdot \int_0^t H(t') dt' \quad (6)$$

where:

- $\epsilon = \frac{\beta}{4+\alpha}$ (dimensionless exponent)
- $H(t) = T_s^{4+\alpha}(t)$ (generalized enthalpy)
- $C(\rho) = \frac{16}{4+\alpha} \frac{g\sigma_{SB}}{3f\rho^{2-\mu+\lambda_p}}$ (material-dependent constant)
- $\alpha, \beta, \mu, \lambda_p$ are opacity and equation-of-state exponents
- f, g are fitting parameters for the material

3.2 Time-Marching Algorithm (Appendix A)

Starting from an initial condition, we march forward in time:

Step 1: Compute flux from previous surface temperature

$$F(0, t_i) = 2\sigma_{SB} [T_D^4(t_i) - T_s^4(t_{i-1})] \quad (7)$$

Step 2: Account for wall energy loss (if applicable)

$$F_{\text{net}}(t_i) = F(0, t_i) - \frac{\dot{E}_{\text{wall}}(t_i)}{A} \quad (8)$$

where $A = \pi R^2$ is the cross-sectional area.

Step 3: Integrate flux to get total energy

$$E(t_i) = E(t_{i-1}) + \frac{1}{2} [F_{\text{net}}(t_i) + F_{\text{net}}(t_{i-1})] \Delta t \quad (9)$$

Step 4: Solve for $H(t_i)$ from implicit equation

$$Z_1 \cdot \left[I(t_{i-1}) + \frac{1}{2} (H(t_{i-1}) + H(t_i)) \Delta t \right] \cdot H(t_i)^\epsilon = E_2 \quad (10)$$

where:

$$Z_1 = f^2 \rho^{2(1-\mu)} (2 + \epsilon)(1 - \epsilon) C(\rho) \quad (11)$$

$$E_2 = \left(\frac{2 + \epsilon}{1 - \epsilon} \right)^2 E(t_i) \quad (12)$$

$$I(t_i) = I(t_{i-1}) + \frac{1}{2} [H(t_{i-1}) + H(t_i)] \Delta t \quad (13)$$

This is solved numerically using Brent's method.

Step 5: Extract surface temperature

$$T_s(t_i) = H(t_i)^{1/(4+\alpha)} \quad (14)$$

Step 6: Compute front position

$$x_F(t_i) = \sqrt{\frac{2 + \epsilon}{1 - \epsilon} \cdot C(\rho) \cdot H(t_i)^{-\epsilon} \cdot I(t_i)} \quad (15)$$

Step 7: Compute axial temperature profile (First-Order HR Profile) For $0 \leq y \leq 1$, where $y = x/x_F(t_i)$, the first-order Hammer–Rosen spatial temperature profile is given by:

$$T(x, t_i) = T_s(t_i) \left[(1 - y) \left(1 + \frac{\epsilon}{2} (1 - A_i) y \right) \right]^{1/[n(1-\epsilon)]} \quad (16)$$

with $T(x, t_i) = 0$ for $y > 1$, where $n = 4 + \alpha$ and the profile parameter $A_i \equiv A(t_i)$ is evaluated using the finite difference estimate of $\dot{H}(t_i)$:

$$A_i = \frac{x_F^2(t_i)}{C(\rho) \cdot H(t_i)^{1-\epsilon}} \frac{\dot{H}(t_i)}{H(t_i)}, \quad \dot{H}(t_i) \approx \frac{H(t_i) - H(t_{i-1})}{\Delta t}. \quad (17)$$

4 Wall Energy Loss and Albedo

4.1 Energy Balance at Wall

The cylindrical wall at $r = R$ absorbs radiation according to:

$$\frac{dE_{\text{wall}}}{dt} = 2\pi R L \sigma_{SB} \left(T_s^4 - \frac{\dot{E}_{\text{wall}}}{4\pi R L \sigma_{SB}} \right) \quad (18)$$

The factor of 2 accounts for flux in both directions (inward and outward).

4.2 Albedo (Wall Reflectivity)

The albedo α_{wall} is defined as the ratio of reflected to incident flux:

$$\alpha_{\text{wall}} = \frac{F_{\text{in}}}{F_{\text{out}}} = \frac{\sigma_{SB}T_s^4 + \frac{\dot{E}_{\text{wall}}}{2}}{\sigma_{SB}T_s^4 - \frac{\dot{E}_{\text{wall}}}{2}} \quad (19)$$

This quantity varies with time as the surface temperature and wall absorption evolve. Typical values:

- $\alpha \approx 0$: Perfect absorber (black wall)
- $\alpha \approx 1$: Perfect reflector (adiabatic wall)
- $0 < \alpha < 1$: Partial absorption (realistic case)

5 Two-Dimensional Extension: Separation of Variables

5.1 Assumed Product Solution

For the cylindrical problem with wall effects, we seek solutions of the form:

$$T(r, z, t) = T_0(z, t) \cdot \Phi(r) \quad (20)$$

where $T_0(z, t)$ is the axial-temporal solution (from 1D Marshak) and $\Phi(r)$ describes radial variation.

5.2 Radial Eigenvalue Problem

The radial function satisfies:

$$\frac{1}{r} \frac{d}{dr} \left(r \frac{d\Phi}{dr} \right) + \kappa^2 \Phi = 0 \quad (21)$$

with boundary conditions:

- Regularity at $r = 0$: $\frac{d\Phi}{dr} \Big|_{r=0} = 0$
- Robin condition at wall: $\frac{d\Phi}{dr} \Big|_{r=R} = -\epsilon \frac{\Phi(R)}{R}$

where $\epsilon = 1 - \alpha_{\text{wall}}$ measures wall absorption.

5.3 Bessel Function Eigenmodes

The general solution is:

$$\Phi(r) = AJ_0(\kappa r) + BY_0(\kappa r) \quad (22)$$

Since $Y_0(0) \rightarrow -\infty$, we require $B = 0$ for regularity. Thus:

$$\Phi(r) = J_0(\kappa r) \quad (23)$$

The boundary condition at $r = R$ gives the transcendental equation:

$$\boxed{\kappa R \cdot J_1(\kappa R) = \epsilon \cdot J_0(\kappa R)} \quad (24)$$

This has infinitely many solutions κ_n for $n = 1, 2, 3, \dots$. We focus on the **fundamental mode** κ_0 (smallest positive root).

5.4 Eigenvalue Solution Strategy

To solve $xJ_1(x) = \epsilon J_0(x)$ where $x = \kappa R$:

1. Define $f(x) = xJ_1(x) - \epsilon J_0(x)$
2. Scan for sign changes in $f(x)$ on $[0, x_{\max}]$ with step dx
3. Use Brent's root-finding algorithm in each bracketing interval
4. Return $\kappa_n = x_n/R$ for $n = 1, 2, \dots, N$

The first root κ_0 gives the slowest-decaying mode and dominates the long-time behavior.

6 Radial Front Position

6.1 Front Definition

The heat front is defined as the isotherm at which $T = T_{\text{threshold}}$ (typically a small fraction of peak temperature). In 1D, this gives $z_F(t)$. In 2D:

$$z_F(r, t) = z_F(t) \cdot \frac{J_0(\kappa_0 r)}{J_0(0)} = z_F(t) \cdot J_0(\kappa_0 r) \quad (25)$$

since $J_0(0) = 1$. This shows:

- At centerline ($r = 0$): $z_F(0, t) = z_F(t)$ (recovers 1D solution)
- At wall ($r = R$): $z_F(R, t) = z_F(t) \cdot J_0(\kappa_0 R)$
- The front is **curved**, with maximum penetration on axis

6.2 Physical Interpretation

- **Wall absorption** ($\epsilon > 0$): Reduces κ_0 , making $J_0(\kappa_0 R)$ larger, so front is less curved
- **Perfect reflector** ($\epsilon \rightarrow 0$): $\kappa_0 R \rightarrow 2.405$ (first zero of J_0), front hits zero at wall
- **Adiabatic limit** ($\epsilon \rightarrow 1$): $\kappa_0 \rightarrow 0$, $J_0(\kappa_0 r) \rightarrow 1$, front becomes flat (1D recovery)

7 Two-Dimensional Temperature Distribution

7.1 Self-Similar Temperature Profile

Beyond determining the front position, we can compute the full temperature field $T(r, z, t)$ using the self-similar solution. The axial temperature profile in the heated region follows:

$$T(z, t) = T_s(t) \left(1 - \frac{z}{z_F(t)} \right)^\gamma \quad (26)$$

where the exponent is:

$$\gamma = \frac{1}{4 + \alpha - \beta} \quad (27)$$

For typical materials (e.g., SiO₂ foam with $\alpha = 3.53$, $\beta = 1.1$):

$$\gamma = \frac{1}{4 + 3.53 - 1.1} = \frac{1}{6.43} \approx 0.156 \quad (28)$$

This profile exhibits:

- Maximum temperature $T_s(t)$ at the irradiated surface ($z = 0$)
- Smooth decay toward the heat front
- Temperature approaches zero at $z = z_F(t)$

7.2 Radially-Dependent Temperature Field

Incorporating the radial structure from the Bessel eigenmode, the full 2D temperature distribution is:

$$T(r, z, t) = T_s(t) \left(1 - \frac{z}{z_F(r, t)} \right)^\gamma \quad \text{for } z < z_F(r, t) \quad (29)$$

where $z_F(r, t) = z_F(t) \cdot J_0(\kappa_0 r)$ is the radially-varying front position. For regions beyond the front:

$$T(r, z, t) = 0 \quad \text{for } z \geq z_F(r, t) \quad (30)$$

7.3 Key Features of the Temperature Field

1. **Hot spot at centerline:** Maximum temperature occurs at ($r = 0, z = 0$) with $T_{\max} = T_s(t)$
2. **Radial cooling:** At fixed z , temperature decreases with r due to:
 - Wall energy absorption
 - Reduced front penetration near wall
 - Bessel function modulation
3. **Axial gradient:** At fixed r , temperature decreases from $T_s(t)$ at $z = 0$ to zero at $z = z_F(r, t)$
4. **Curved isotherms:** Surfaces of constant temperature follow curves in (r, z) space
5. **Energy content:** Total energy in foam can be computed by integrating:

$$E_{\text{foam}} = 2\pi \int_0^R \int_0^{z_F(r, t)} a T(r, z, t)^4 r \, dz \, dr \quad (31)$$

7.4 Temperature Heatmap Visualization

The 2D temperature field is visualized using contour plots (heatmaps) in the (r, z) plane:

Implementation:

1. Create mesh grid: $r_i \in [0, R]$ with $N_r = 100$ points, $z_j \in [0, L]$ with $N_z = 200$ points
2. For each (r_i, z_j) :
 - Compute local front position $z_F(r_i, t)$ using Bessel interpolation
 - If $z_j < z_F(r_i, t)$: $T_{ij} = T_s(t) (1 - z_j/z_F(r_i, t))^\gamma$
 - If $z_j \geq z_F(r_i, t)$: $T_{ij} = 0$
3. Plot using `matplotlib.pyplot.contourf` with colormap (e.g., 'Spectral_r')
4. Overlay front curve $z_F(r, t)$ as boundary line
5. Add contour lines to show temperature levels

Colormap interpretation:

- Red/yellow regions: High temperature (near surface)
- Green/cyan regions: Intermediate temperature
- Blue regions: Low temperature (near front) or unheated (beyond front)

8 Computational Implementation

8.1 Eigenvalue Calculation

Module: `eigen_bessel_solver.py`

Key function:

`kappa_roots(eps, R, n_roots, x_max=None, dx=1e-2)`

Algorithm:

1. Compute $\epsilon = 1 - \alpha_{\text{wall}}$ from current albedo
2. Scan $f(x) = xJ_1(x) - \epsilon J_0(x)$ for sign changes
3. Use `scipy.optimize.brentq` to find roots
4. Return $\kappa_n = x_n/R$

8.2 Time-Marching with Bessel Data Storage

Module: `model_main.py`

Key function:

```
_marshak_appendixA_march(times_to_store, wall_loss=True, ...)
```

At each time step t_i where $i > 1$ and `wall_loss=True`:

1. Compute albedo $\alpha(t_i)$ from surface temperature and wall energy loss rate
2. Compute $\epsilon = \frac{3}{4}(1 - \alpha(t_i))\frac{R}{\lambda_{\text{Ross}}}$
3. Solve for κ_0 using eigenvalue solver
4. Compute $J_0(\kappa_0 r_j)$ for radial grid $r_j \in [0.01, R]$ with $N_r = 100$ points
5. Store in dictionary:

```
bessel_data[t_ns] = {  
    'r_grid': r_grid,  
    'z_grid': z_grid,  
    'J0_profiles': J0_profiles, # shape (Nz, Nr)  
    'kappa_0': kappa_0,  
    'epsilon': epsilon,  
    'albedo': albedo,  
    'lambda_ross': lambda_ross  
}
```

Implementation note (current code):

- For runs with `wall_loss=False`, `bessel_data` can be empty.
- The plotting routine uses a fallback in this case:
 - set $t_{\text{closest}} = t_{\text{target}}$,
 - build a radially flat $z_F(r, t)$ profile from 1D $z_F(t)$,
 - continue plotting without nearest-time lookup in an empty dictionary.

8.3 Wall-Profile Construction in Heatmaps

Module: `shape_2D_analytical_model.py`

The helper `_compute_wall_heyney_horizontal_profile` now distinguishes two explicit modes:

1. **Ablation mode:** profile starts at local ablation interface $R_{\text{ablation}}(z)$ and ends at local stop radius $r_{\text{stop}}(z)$.
2. **Non-ablation mode:** profile starts at fixed foam boundary R_{cm} and ends at $R_{\text{cm}} + \Delta r_{\text{pen}}(z)$ (or equivalent absolute radius profile).

In both modes, the Henyey coordinate is evaluated pointwise along wall cells:

$$\eta(r, z) = \frac{r - r_{\text{start}}(z)}{r_{\text{stop}}(z) - r_{\text{start}}(z)} \quad (32)$$

Also, unreached/cold gold cells are set to $T = 0$ in the plotting mesh (not NaN), preventing white masked regions in the heatmap.

8.4 Foam-Gold Region Differentiation and Temperature Assignment

Module: `shape_2D_analytical_model.py`

In the heatmap implementation, the cylindrical cross-section is split into two material regions:

- **Foam region:** $r \leq R_{\text{cm}}$
- **Gold region:** $r > R_{\text{cm}}$

The foam temperature is first computed on the foam radial mesh via the self-similar law:

$$T_{\text{foam}}(r, z, t) = \begin{cases} T_s(t) \left(1 - \frac{z}{z_F(r, t)}\right)^\gamma, & z < z_F(r, t), \\ 0, & z \geq z_F(r, t). \end{cases} \quad (33)$$

Implementation detail: the 2D plotting grid is built early in the plotting routine from the current radial and axial meshes, i.e. $(R_{\text{mesh}}, Z_{\text{mesh}}) = \text{meshgrid}(r_{\text{mesh}}, z_{\text{mesh}})$, and then the same grid is reused for temperature filling, contour extraction, and final rendering.

Then the plotting mesh is initialized as zeros and filled in two stages:

1. Interpolate T_{foam} onto all cells with $r \leq R_{\text{cm}}$.
2. Overwrite gold-side cells with the wall Henyey-like profile where valid.

The wall profile is written only on the radial interval between a start radius and stop radius:

$$r_{\text{start}}(z) = \begin{cases} R_{\text{ablation}}(z), & \text{ablation mode,} \\ R_{\text{cm}}, & \text{non-ablation mode,} \end{cases} \quad r_{\text{stop}}(z) = R_{\text{cm}} + \Delta r_{\text{pen}}(z) \quad (34)$$

Here, $\Delta r_{\text{pen}}(z)$ (often referenced in code as `r_pen`) is implemented from the snapshot field `wall_penetration_radius_profile`. The implementation supports two input conventions:

- **Absolute-radius convention:** profile already stores $r_{\text{stop}}(z)$.
- **Depth convention:** profile stores penetration depth into gold, then code converts it to $r_{\text{stop}}(z) = R_{\text{cm}} + \Delta r_{\text{pen}}(z)$.

For robustness, the profile is sanitized before use: if shape mismatch is detected, a fallback profile with $r_{\text{stop}}(z) = R_{\text{cm}}$ is used; otherwise each value is clipped to the plotting radial domain so the wall profile remains bounded and physically consistent.

with normalized coordinate

$$\eta(r, z) = \frac{r - r_{\text{start}}(z)}{r_{\text{stop}}(z) - r_{\text{start}}(z)} \quad (35)$$

and wall temperature

$$T_{\text{gold}}(r, z, t) = T_{\text{local}}(z, t) (1 - \eta(r, z))^{\gamma_{\text{wall}}}, \quad 0 \leq \eta \leq 1. \quad (36)$$

Implementation-wise, this is controlled by masks/profile arrays stored in each time snapshot (e.g., `ablation_foam_mask`, `ablation_wall_mask`, `wall_penetration_radius_profile`), and the final plotted field is assembled as:

$$T_{\text{plot}} = T_{\text{foam}} \text{ in foam domain,} \quad T_{\text{plot}} = T_{\text{gold}} \text{ where wall profile is valid,} \quad (37)$$

with all remaining cells left at 0.

8.5 Shock Front in the Wall Layer

As an another nice addition to the 2D model, we can compute the shock front position in the wall layer using the same time-dependent surface temperature history that drives the wall-loss model.

The shock starts from the already-penetrated wall region and advances outward of the holraum as energy accumulates.

For each axial location z , the code computes:

$$r_{\text{shock}}(z, t) = r_{\text{wall}}(z, t) + r_{\text{shock}}(z, t) \quad (38)$$

where:

- $r_{\text{wall}}(z, t)$ is the wall penetration front profile,
- $r_{\text{shock}}(z, t)$ is the local shock penetration depth,
- both quantities are stored as one-dimensional arrays over the axial (z) grid.

s

Profile Construction

The shock profile is computed using the same time-dependent surface temperature history that drives the wall-loss model. For each z_i behind the heated region:

1. Compute the pressure by using the formula $P(z_i) = 2.71T_0(z)^{2.63}$, where $T_0(z)$ is the gold-foam interface temperature at the time when the shock starts to heat that location.
2. Compute the local exposure time $t_{\text{exposed}}(z_i) = t - t_{\text{heat}}(z_i)$.
3. Evaluate the shock penetration depth from the local thermodynamic state using $m_s = 5.38 \cdot 10^{-3} P_0^{0.5} \cdot t_{\text{exposed}}$ and $r_{\text{shock}}(z_i) = m_s / \rho_{\text{gold}}$.
4. Convert that depth into an absolute radius profile by adding it to the wall front.

Implementation Notes

The current code stores the shock as 1D profiles such as:

- `shock_penetration_depth_cm_profile`
- `shock_penetration_radius_profile`

This keeps the shock handling consistent with the wall-front logic and avoids introducing an extra radial modulation factor.

8.6 Radial Front Reconstruction

Module: `comparison.py`

Key function:

`plot_radial_front_profiles(bessel_data, z_F_array, times_array, times_ns)`

For each requested time t :

1. Find closest available time in `bessel_data`
2. Get 1D front position $z_F(t)$ from analytical solution
3. Interpolate J_0 profiles at $z = z_F(t)$ to get $J_0(\kappa_0 r_j, z_F)$
4. Normalize: $J_0^{\text{norm}}(r) = J_0(\kappa_0 r, z_F) / J_0(0, z_F)$
5. Compute: $z_F(r, t) = z_F(t) \cdot J_0^{\text{norm}}(r)$

Visualization:

- **1D plot:** $z_F(r, t)$ vs r at fixed times (shows radial structure)
- **2D spatial plot:** (r, z) coordinates with curved front and shaded heated region

9 Results and Visualization

9.1 Output Files

1. **Front comparison:** `front_position_marshall_vs_nonmarshall.png`
 - Compares 1D solutions with/without Marshak BC
 - Shows effect of wall loss on centerline position
2. **2D spatial view:** `2D_front_spatial.png`
 - (r, z) coordinate plot with domain $[0, R] \times [0, L]$
 - Curved front position as red line
 - Heated region shaded in orange
 - Boundary markers for $r = R$ (wall) and $z = L$ (far end)
3. **Temperature heatmaps:** `temperature_heatmap_2D.png`

- 3-panel contour plot showing $T(r, z, t)$ distribution
- Times: 1 ns, 2 ns, 2.5 ns
- Colormap: Spectral_r (blue=cold, red=hot)
- Displays self-similar temperature profile with exponent $\gamma = 1/(4 + \alpha - \beta)$
- Overlay of curved front position
- Contour lines indicating temperature levels
- Shows spatial structure of heated region

9.2 Typical Behavior

1. Early time ($t \ll 1$ ns):

- Front moves slowly, stays near $z = 0$
- Albedo $\alpha \approx 0$ (wall is cold, absorbs strongly)
- Large $\epsilon \Rightarrow$ smaller $\kappa_0 \Rightarrow$ flatter front

2. Intermediate time ($t \sim 1 - 2$ ns):

- Front accelerates, reaches $z_F \sim 0.05 - 0.1$ cm
- Albedo increases as wall heats up
- Front curvature becomes more pronounced

3. Late time ($t > 2.5$ ns):

- Front slows due to energy loss to wall
- Albedo saturates near steady value
- Radial structure stabilizes

10 Code Structure

10.1 Module Organization

1. `parameters.py`

- Material-specific constants ($\alpha, \beta, f, g, \rho, R, L$)
- Drive temperature data loading
- Grid discretization (z -grid, time step)

2. `eigen_bessel_solver.py`

- Function: `kappa_roots(eps, R, n_roots)`
- Solves $xJ_1(x) = \epsilon J_0(x)$ for eigenvalues
- Returns $\kappa_n = x_n/R$

3. `model_main.py`

- Time-marching with Marshak BC: `_marshak_appendixA_march`
- Energy partition: `compute_wall_energy_loss`
- Albedo calculation: `compute_albedo_step`
- Front position: self-similar scaling laws
- Bessel data storage at each time step

4. `shape_2D_analytical_model.py`

- 2D front and temperature plotting routines
- Empty-bessel_data fallback for non-wall-loss runs
- Ablation/non-ablation wall profile branching
- Gold region initialized with $T = 0$ for robust rendering

5. `comparison.py`

- Main plotting function: `plot_both_marshak_and_nonmarshak_heat_fronts`
- Radial front profiles: `plot_radial_front_profiles`
- 2D spatial visualization: `plot_2D_front_spatial`
- Temperature heatmaps: `plot_temperature_heatmap_2D`
- Comparison with experimental data

10.2 Data Flow

```

parameters.py (material constants)
|
v
model_main.py (time marching)
|---> compute T_s(t), z_F(t), E_wall(t), albedo(t)
|---> solve for kappa_0(t) using eigen_bessel_solver.py
|---> store J_0(kappa_0 r) profiles in bessel_data dict (wall_loss=True)
|
v
shape_2D_analytical_model.py (2D reconstruction)
|---> if bessel_data is empty: use flat-radial z_F fallback
|---> wall profile:
|       ablation      : R_ablation(z) -> r_stop(z)
|       no ablation   : R_cm -> R_cm + penetration(z)
|---> initialize cold gold cells with T=0
|
v
comparison.py (orchestration)
|---> read z_F(t), T_s(t), and bessel_data
|---> compute z_F(r,t) = z_F(t) * J_0(kappa_0 r)
|---> compute T(r,z,t) = T_s(t) * (1 - z/z_F(r,t))^gamma
|---> generate plots:
|       - 1D front profiles vs r
|       - 2D spatial view with curved front
|       - 2D temperature heatmaps

```

11 Extensions and Future Work

11.1 Higher-Order Modes

The current implementation uses only the fundamental mode κ_0 . A more complete solution would include:

$$z_F(r, t) = \sum_{n=0}^{\infty} A_n(t) J_0(\kappa_n r) \quad (39)$$

where coefficients $A_n(t)$ are determined by initial conditions and evolve according to modal diffusion equations.

11.2 Time-Dependent Albedo

Currently, $\alpha(t)$ varies with time as the wall heats up. Future refinements:

- Multi-layer wall model (foam-wall interface)
- Temperature-dependent wall properties
- Non-local energy deposition in wall

11.3 Ablation Effects

When wall ablation occurs, $R(t)$ decreases with time:

$$\frac{dR}{dt} = - \frac{\dot{E}_{\text{wall}}}{2\pi R L \cdot \rho_{\text{wall}} \cdot E_{\text{ablation}}} \quad (40)$$

This introduces time-dependent geometry, requiring:

- Recomputation of eigenvalues at each time step
- Moving boundary formulation
- Mass conservation tracking

11.4 Effective Lambda Correction

In confined geometries, the effective mean free path is limited by geometry:

$$\lambda_{\text{eff}} = (\lambda_{\text{ross}}^{-p} + \lambda_{\text{geom}}^{-p})^{-1/p} \quad (41)$$

where $\lambda_{\text{geom}} = 2R$ and $p \sim 1.5 - 2.0$. This modifies the diffusion coefficient and front speed.

12 Validation and Benchmarking

12.1 Comparison with Experiments

The code has been validated against:

1. **Back et al.** (Article 1): SiO₂ foam experiments at OMEGA
2. **Massen et al.** (Article 2): Doped foam experiments with varying drive temperatures
3. **Moore et al.:** Alternative foam compositions

Typical agreement: 5-15% in front position, 10-20% in surface temperature, depending on material uncertainties.

12.2 Code-to-Code Comparison

Results have been cross-checked with:

- Full 2D radiation-hydrodynamics simulations (Hydra, HELIOS)
- 1D Lagrangian codes with grey diffusion
- Independent semi-analytic models from collaborators

12.3 Convergence Tests

Numerical convergence verified by:

1. Time step refinement: $\Delta t \rightarrow 0$ (front position converges to $< 1\%$)
2. Radial grid refinement: $N_r = 50, 100, 200$ (Bessel profiles stable)
3. Eigenvalue solver tolerance: $dx = 10^{-2}, 10^{-3}, 10^{-4}$ (kappa accurate to 6 digits)

13 Conclusion

This document has presented a complete analytical-numerical framework for computing two-dimensional heat front positions in cylindrical radiation diffusion with wall effects. The key innovations are:

1. **Separation of scales:** 1D Marshak solution for axial propagation + Bessel eigenmode for radial structure
2. **Dynamic eigenvalues:** $\kappa_0(t)$ computed from time-dependent albedo
3. **Energy accounting:** Explicit tracking of wall absorption and its feedback on front motion
4. **Efficient storage:** Bessel profiles stored at each time step enable rapid post-processing

The resulting model provides:

- Predictions of 3D front shape: $z_F(r, t)$
- Full 2D temperature distribution: $T(r, z, t)$ with self-similar profiles
- Quantification of wall effects through albedo $\alpha(t)$
- Energy partition between foam heating and wall loss
- Visualization tools for (r, z) spatial structure including heatmaps

Applications include design of indirect-drive ICF targets, X-ray ablation studies, and validation of multi-dimensional radiation transport codes.

A Derivation of Eigenvalue Equation

Starting from the steady-state diffusion equation in cylindrical coordinates:

$$\frac{1}{r} \frac{d}{dr} \left(r \frac{d\Phi}{dr} \right) + \kappa^2 \Phi = 0 \quad (42)$$

Multiplying by r :

$$\frac{d}{dr} \left(r \frac{d\Phi}{dr} \right) + \kappa^2 r \Phi = 0 \quad (43)$$

This is Bessel's equation of order zero. The general solution is:

$$\Phi(r) = A J_0(\kappa r) + B Y_0(\kappa r) \quad (44)$$

Since $Y_0(r) \rightarrow -\infty$ as $r \rightarrow 0$, we require $B = 0$ for finite solution at origin:

$$\Phi(r) = J_0(\kappa r) \quad (45)$$

At the wall ($r = R$), the Robin boundary condition relates flux to value:

$$-D \frac{\partial T}{\partial r} \Big|_{r=R} = h(T(R) - T_{\text{ext}}) \quad (46)$$

For large temperature difference, $T_{\text{ext}} \approx 0$ and $h \propto (1 - \alpha)\sigma T^3$:

$$\frac{d\Phi}{dr} \Big|_{r=R} = -\frac{h}{D} \Phi(R) \equiv -\epsilon \frac{\Phi(R)}{R} \quad (47)$$

where $\epsilon = hR/D = (1 - \alpha) \times (\text{geometric factors})$.

Taking derivative of $\Phi(r) = J_0(\kappa r)$:

$$\frac{d\Phi}{dr} = -\kappa J_1(\kappa r) \quad (48)$$

At $r = R$:

$$-\kappa J_1(\kappa R) = -\epsilon \frac{J_0(\kappa R)}{R} \quad (49)$$

Simplifying:

$$\boxed{\kappa R \cdot J_1(\kappa R) = \epsilon \cdot J_0(\kappa R)} \quad (50)$$

This transcendental equation has infinitely many positive roots κ_n for $n = 1, 2, 3, \dots$

B Numerical Solution of Transcendental Equation

Define $x = \kappa R$ and $f(x) = xJ_1(x) - \epsilon J_0(x)$. We seek zeros of $f(x)$.

Algorithm:

1. Choose search range $[0, x_{\max}]$ where $x_{\max} \sim (N + 2)\pi$ for N roots
2. Discretize with step dx (e.g., $dx = 0.01$)
3. Compute $f(x_i)$ at each grid point
4. Identify sign changes: $f(x_i) \cdot f(x_{i+1}) < 0$
5. Refine each bracket using Brent's method:

```
from scipy.optimize import brentq
root = brentq(f, x_i, x_{i+1}, args=(epsilon,))
```

6. Return $\kappa_n = x_n/R$ for each root

Accuracy: Brent's method converges super-linearly with typical tolerance 10^{-12} in 10-20 iterations.

C Bessel Function Properties

C.1 Key Identities

1. $J_0(0) = 1$
2. $J'_0(x) = -J_1(x)$
3. $\int_0^R r J_0(\kappa r) dr = \frac{R}{\kappa} J_1(\kappa R)$
4. Orthogonality:

$$\int_0^R r J_0(\kappa_m r) J_0(\kappa_n r) dr = \frac{R^2}{2} J_0^2(\kappa_n R) \delta_{mn} \quad (51)$$

C.2 Asymptotic Behavior

For large x :

$$J_0(x) \sim \sqrt{\frac{2}{\pi x}} \cos\left(x - \frac{\pi}{4}\right) \quad (52)$$

This oscillatory decay explains the multiple eigenvalues κ_n spaced approximately π/R apart.